



SWEN 221

A surreal, dreamlike landscape. In the center, a person stands on a small, dark, rocky outcrop that appears to be part of a vast, ethereal sea of clouds. The person is looking out over a vast, colorful sky. The sky is filled with large, billowing clouds in shades of pink, purple, blue, and yellow. A bright, glowing light source, possibly a sun or moon, is visible in the distance, casting a warm glow over the scene. The sky is also filled with stars, a comet streaking across the upper left, and a large, colorful nebula or galaxy in the upper right. The overall atmosphere is one of wonder and exploration.

Marco Servetto
CO258

www.youtube.com/MarcoServetto





Unit tests

Unit testing with JUnit

JUnit a Unit Testing Framework

- **Kent Beck** (XP, Smalltalk)
- **Erich Gamma** (Eclipse, Patterns)

Using Junit:

- Tests are Java methods
- Test suites are Java classes
- Annotations mark them out
- API for writing tests
- IDE support (Eclipse...)



Anatomy of a **JU**nit Test

In your test class

- (typically 1-1 with top level modeling concepts)

```
import static org.junit.jupiter.api.Assertions.*;  
import org.junit.jupiter.api.Test;
```

- Annotate methods with **@Test**

The JUnit API

A range of assert** methods:

- assertTrue(boolean)
- assertTrue(String message, boolean)
- ◆ assertEquals(Object expect, Object actual) //has graphical support for string diffs!!

- assertEquals(float expected, float actual, float delta)

- assertNull, assertNotNull

- assertFalse

- assertEquals, assertEqualsNotSame

- ◆ assertThrows(Err.class,()->myCode)

- fail(), fail(String message)

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
```

```
public class MyTestSuit{
    @Test public void testDogBarks(){
        Dog dog = new Dog(...);
        String sound = dog.bark();
        assertEquals("Woof", sound);
    }
}
```

Example:

how to test this code

```
record MyDate(int day, int month, int year){
```

```
    MyDate{
```

```
        // check invariants hold
```

```
        if (day <= 0 || month < 0){ throw new IllegalArgumentException(..); }
```

```
        if (day > 31 || month > 12){ throw new IllegalArgumentException(..); }
```

```
        var aprJunSepNov= month == 4 || month == 6 || month == 9 || month == 11;
```

```
        if (aprJunSepNov && day > 30){ throw new IllegalArgumentException(..); }
```

```
        var badFeb= month == 2
```

```
            && (day > 29 || (day > 28 && !isLeapYear(year)));
```

```
        if (badFeb){ throw new IllegalArgumentException(..); }
```

```
    }
```

```
    private static boolean isLeapYear(int year){
```

```
        //Every year that is exactly divisible by four is a leap year,
```

```
        //except for years that are exactly divisible by 100,
```

```
        //but these centurial years are leap years
```

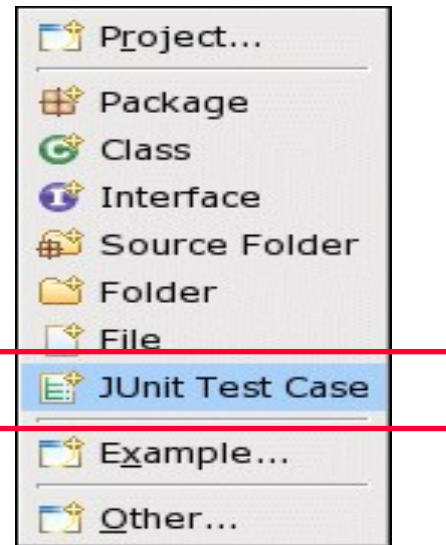
```
        //if they are exactly divisible by 400
```

```
        return year % 4 == 0 && (year % 100 != 0 || year % 400 == 0);
```

```
    }
```

```
}
```


Starting JUnit



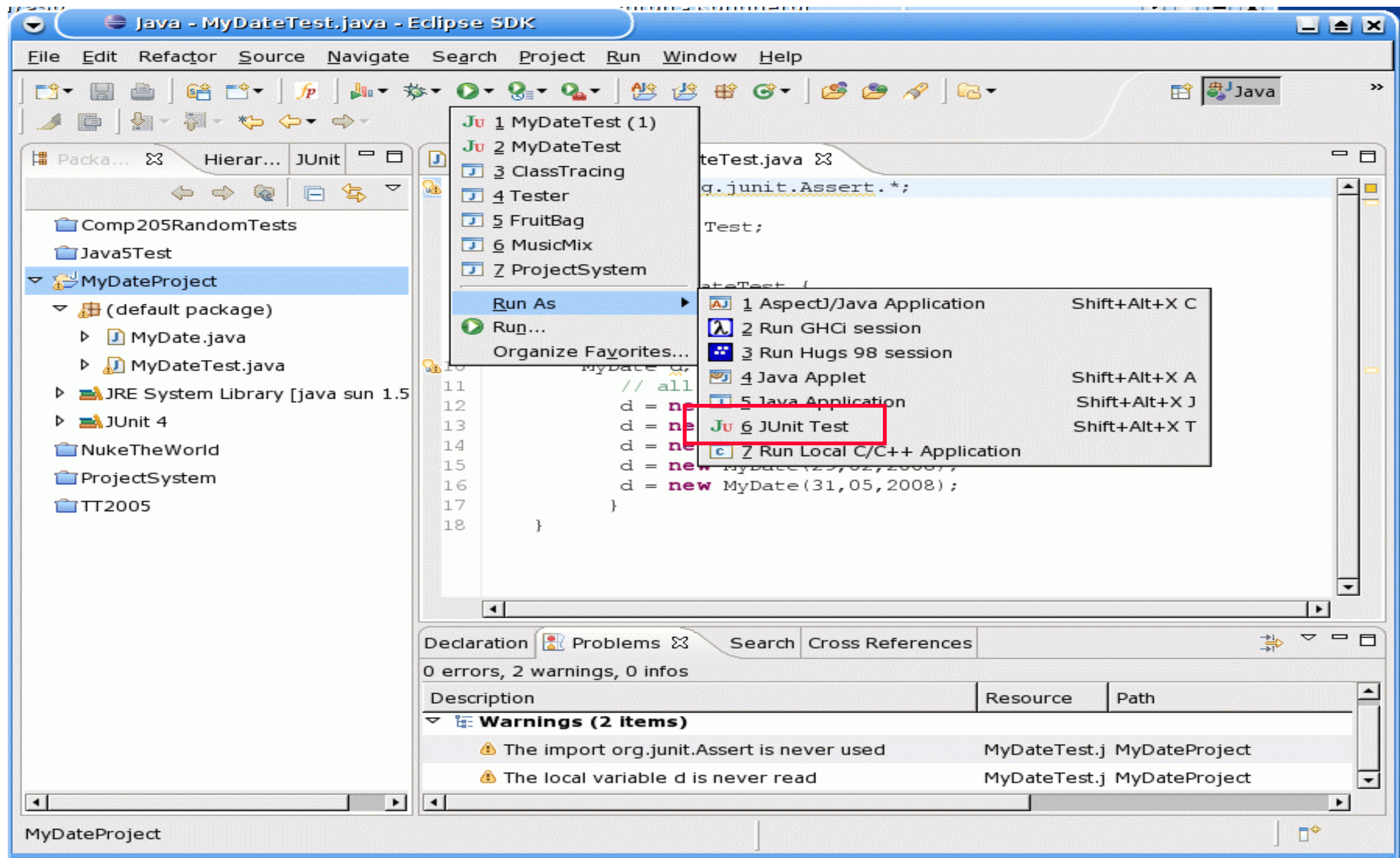
A screenshot of the 'New JUnit Test Case' dialog box. The dialog has a title bar with a question mark icon, a minimize button, a maximize button, and a close button. The main content area is titled 'JUnit Test Case' and contains the following elements:

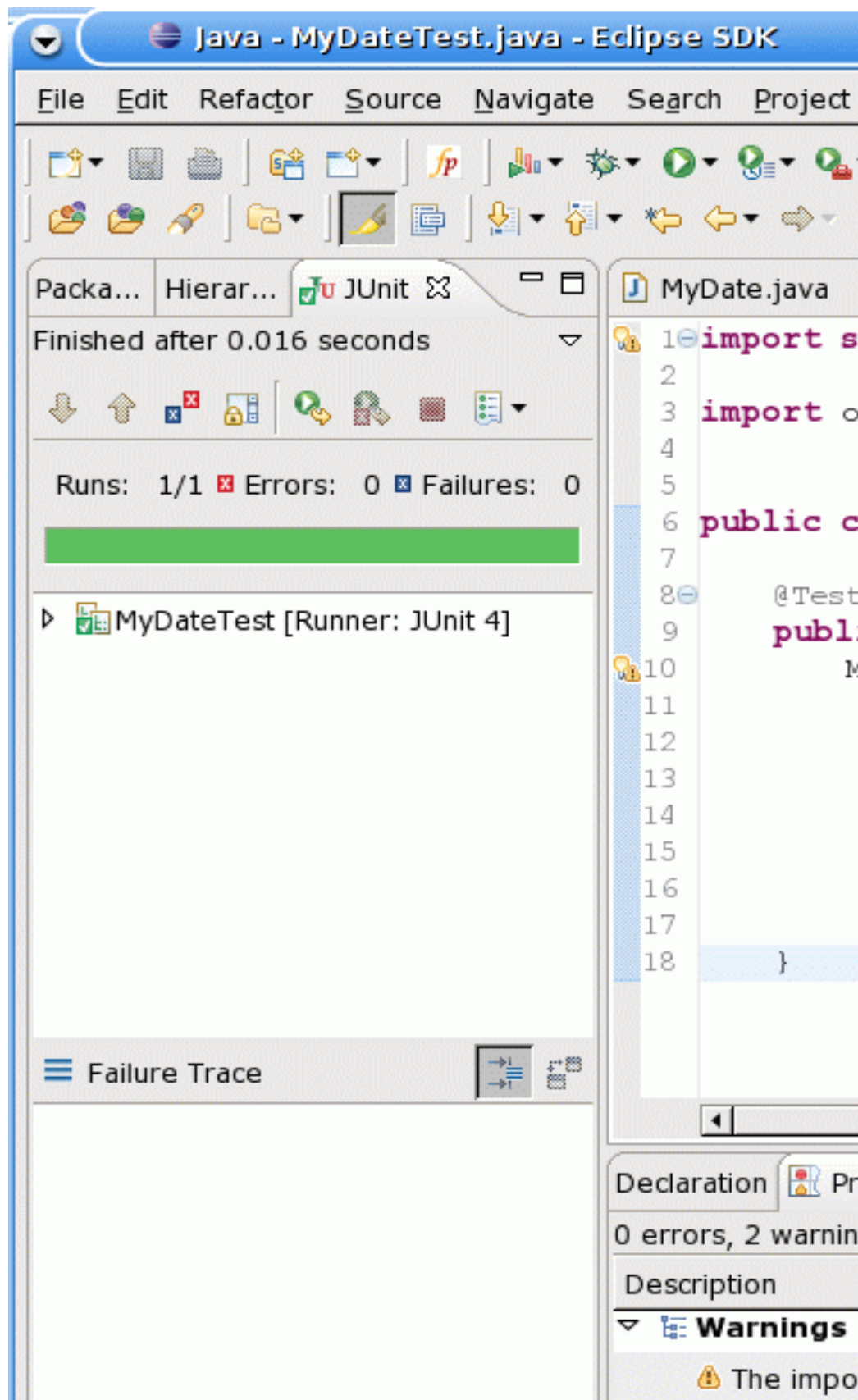
- A text box with the instruction: 'Select the name of the new JUnit test case. You have the options to specify the class under test and on the next page, to select methods to be tested.'
- Three radio buttons for selecting the type of test case:
 - ☐ New JUnit 3 test
 - ☐ New JUnit 4 test
 - ☒ New JUnit Jupiter test (This option is highlighted with a red rectangular box.)
- Fields for 'Source folder:' (containing '/src') and 'Package:' (containing 'gg'), each with a 'Browse...' button.
- A 'Name:' field containing 'MyDateTest'.
- A 'Superclass:' field containing 'java.lang.Object' with a 'Browse...' button.
- A section titled 'Which method stubs would you like to create?' with four checkboxes:
 - ☐ setUpBeforeClass()
 - ☐ tearDownAfterClass()
 - ☐ setUp()
 - ☐ tearDown()
- A checkbox for 'constructor'.
- A section titled 'Do you want to add comments? (Configure templates and default value [here](#))' with a checkbox for 'Generate comments'.
- A 'Class under test:' field containing 'gg.MyDate' with a 'Browse...' button.

At the bottom of the dialog, there is a row of buttons: a question mark icon, '< Back', 'Next >', 'Cancel', and 'Finish'.

A simple JUnit test

```
public class MyDateTest{
    @Test public void testConstructValidDate(){
        // all these dates are right!
        new MyDate( 1, 1, 1);
        new MyDate( 1, 5, 2006);
        new MyDate(28, 2, 2006);
        new MyDate(29, 2, 2008);
        new MyDate(31, 5, 2008);
    }
}
```





Testing the Happy Path

```
public class MyDateTest{  
    @Test public void testConstructValidDate(){  
        // all these dates are right!  
        new MyDate( 1, 1, 1);  
        new MyDate( 1, 5, 2006);  
        new MyDate(28, 2, 2006);  
        new MyDate(29, 2, 2008);  
        new MyDate(31, 5, 2008);  
    }  
}
```


Testing the Unhappy Path

The screenshot shows the Eclipse IDE interface. The top toolbar includes icons for File, Edit, Refactor, Source, Navigate, Search, and Project. Below the toolbar is a package explorer showing a hierarchy of packages and classes. The main editor displays the source code of MyDateTest.java. The code includes a test method testConstructInvalidDate() which uses assertEquals to verify that certain dates are invalid. The test method is currently failing, as indicated by the red 'x' icon in the package explorer and the 'Failure Trace' window at the bottom. The failure trace shows a java.lang.AssertionError: Invalid date di at MyDateTest.testConstructInvalidDate. The bottom status bar indicates '0 errors, 2 warnings'.

```
17 }
18 @Test
19 public void testConstructInvalidDate(){
20     // all these dates are wrong!
21     var re= RuntimeException.class;
22     assertEquals(re, ()->new MyDate(0,0,0) );
23     assertEquals(re, ()->new MyDate(1,0,0) );
24     assertEquals(re, ()->new MyDate(32, 1, 1) );
25     assertEquals(re, ()->new MyDate(29, 2, 2006) );
26     assertEquals(re, ()->new MyDate(31, 9, 2006) );
27     assertEquals(re, ()->new MyDate(31, 4, 2006) );
28     assertEquals(re, ()->new MyDate(31, 6, 2006) );
29     assertEquals(re, ()->new MyDate( 1, 1, -1) );
30 }
31
32 fo
33
34 }
35
36
37 }
```

```
@Test public void testConstructInvalidDate(){
    // all these dates are wrong!
    var re= RuntimeException.class;
    assertEquals(re, ()->new MyDate(0,0,0) );
    assertEquals(re, ()->new MyDate(1,0,0) );
    assertEquals(re, ()->new MyDate(32, 1, 1) );
    assertEquals(re, ()->new MyDate(29, 2, 2006) );
    assertEquals(re, ()->new MyDate(31, 9, 2006) );
    assertEquals(re, ()->new MyDate(31, 4, 2006) );
    assertEquals(re, ()->new MyDate(31, 6, 2006) );
    assertEquals(re, ()->new MyDate( 1, 1, -1) );
}
```

Why?

```
record MyDate(int day, int month, int year){
```

```
    MyDate{
```

```
        // check invariants hold
```

```
        if (day <= 0 || month < 0){ throw new IllegalArgumentException(..); }
```

```
        if (day > 31 || month > 12){ throw new IllegalArgumentException(..); }
```

```
        var aprJunSepNov= month == 4 || month == 6 || month == 9 || month == 11;
```

```
        if (aprJunSepNov && day > 30){ throw new IllegalArgumentException(..); }
```

```
        var badFeb= month == 2
```

```
            && (day > 29 || (day > 28 && !isLeapYear(year)));
```

```
        if (badFeb){ throw new IllegalArgumentException(..); }
```

```
    }
```

```
    private static boolean isLeapYear(int year){
```

```
        //Every year that is exactly divisible by four is a leap year,
```

```
        //except for years that are exactly divisible by 100,
```

```
        //but these centurial years are leap years
```

```
        //if they are exactly divisible by 400
```

```
        return year % 4 == 0 && (year % 100 != 0 || year % 400 == 0);
```

```
    }
```

```
}
```


Why?

```
record MyDate(int day, int month, int year){
```

```
    MyDate{
```

```
        // check invariants hold
```

```
        if (day <= 0 || month < 0){ throw new IllegalArgumentException(..); }
```

```
        if (day > 31 || month > 12){ throw new IllegalArgumentException(..); }
```

```
        var aprJunSepNov= month == 4 || month == 6 || month == 9 || month == 11;
```

```
        if (aprJunSepNov && day > 30){ throw new IllegalArgumentException(..); }
```

```
        var badFeb= month == 2
```

```
            && (day > 29 || (day > 28 && !isLeapYear(year)));
```

```
        if (badFeb){ throw new IllegalArgumentException(..); }
```

```
    }
```

```
    private static boolean isLeapYear(int year){
```

```
        //Every year that is exactly divisible by four is a leap year,
```

```
        //except for years that are exactly divisible by 100,
```

```
        //but these centurial years are leap years
```

```
        //if they are exactly divisible by 400
```

```
        return year % 4 == 0 && (year % 100 != 0 || year % 400 == 0);
```

```
    }
```

```
}
```

How does JUnit works?

- Where is the main? We know there must be a main to start a Java program, right?
- Yes, the main is inside the Junit library.
- That main is receiving the name of the test class/classes in input and is using reflection to run the test methods.
- Then the IDE takes the output of that main and renders it nicely.

Test structure

- An imperative test is divided in 3 main parts:
 - (1) Initialize the data
 - (2) Run the operation
 - (3) Check that the result satisfy our expectations
- So, code is testable if:
 - It is easy to create objects to tests
 - It is easy to run operations on those objects, no need to be inside the full system
 - It is easy to check the result of the operations

Test structure

- Code is testable if:
 - It is easy to create objects to tests
 - It is easy to run operations on those objects, no need to be inside the full system
 - It is easy to check the result of the operations
- Hazards to testability (AVOID):
 - To create this object you need many other objects
 - To create this object, certain static fields in various classes must be set to be 'just right'
 - To run this operation is not enough to pass parameters: some files needs to be in place on the file system, a database needs to be available, ...
 - The result of this operation does not just depend on the input, or it contains some intrinsically non deterministic components.

Exercise

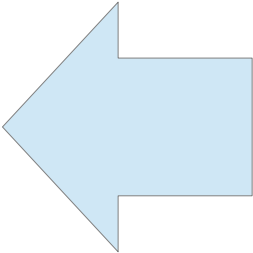
- We are now going to see some code sorting a list with the simple 'selection sort' algorithm.
The algorithm sorts the list left to right, selecting the lowest element from the unsorted portion of the list and swapping it with the first element of the unsorted portion.

Sorting, as in high school programming

```
class Sorter{
    void sort(List<Double> ds){
        for (int i= 0; i < ds.size() - 1; i++){
            int jMin= i;
            for (int j= i + 1; j < ds.size(); j++){
                if (ds.get(j) < ds.get(jMin)){ jMin = j; }
            }
            double tmp= ds.get(jMin);
            ds.set(jMin,ds.get(i));
            ds.set(i,tmp);
        }
    }
}
```

We can improve it, by method decomposition

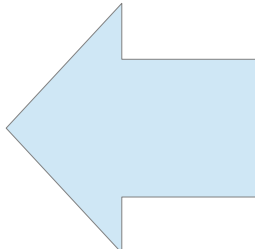
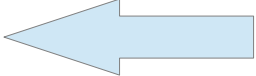
```
class Sorter{  
    void sort(List<Double> ds){  
        for (int i= 0; i < ds.size() - 1; i++){  
            int jMin= i;  
            for (int j= i + 1; j < ds.size(); j++){  
                if (ds.get(j) < ds.get(jMin)){ jMin = j; }  
            }  
            double tmp= ds.get(jMin);  
            ds.set(jMin,ds.get(i));  
            ds.set(i,tmp);  
        }  
    }  
}
```



This is a SWAP

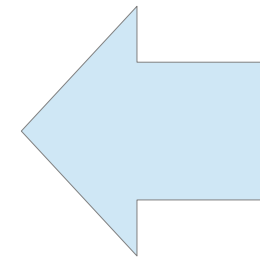
We can improve it, by method decomposition

```
class Sorter{  
    void sort(List<Double> ds){  
        for (int i= 0; i < ds.size() - 1; i++){  
            int jMin= i;  
            for (int j= i + 1; j < ds.size(); j++){  
                if (ds.get(j) < ds.get(jMin)){ jMin = j; }  
            }  
            swap(i,jMin,ds);  
        }  
    }  
    void swap(int i, int j, List<Double> ds) {  
        double tmp= ds.get(i);  
        ds.set(i,ds.get(j));  
        ds.set(j, tmp);  
    }  
}
```



We can improve it, by method decomposition

```
class Sorter{
    void sort(List<Double> ds){
        for (int i= 0; i < ds.size() - 1; i++){
            int jMin= i;
            for (int j= i + 1; j < ds.size(); j++){
                if (ds.get(j) < ds.get(jMin)){ jMin = j; }
            }
            swap(i,jMin,ds);
        }
    }
    void swap(int i, int j, List<Double> ds) {
        double tmp= ds.get(i);
        ds.set(i,ds.get(j));
        ds.set(j, tmp);
    }
}
```



This finds the minIndex

Sorting, with sub methods

```
class Sorter{
    void sort(List<Double> ds){
        for (int i= 0; i < ds.size() - 1; i++){
            swap(i,minIndex(i,ds),ds);
        }
    }
    int minIndex(int i, List<Double> ds){
        int res= i;
        for (int j= i + 1; j < ds.size(); j++){
            if (ds.get(j) < ds.get(res)){ res = j; }
        }
        return res;
    }
    void swap(int i, int j, List<Double> ds) {
        double tmp= ds.get(i);
        ds.set(i,ds.get(j));
        ds.set(j, tmp);
    }
}
```

- Now in addition to testing 'sort', we can also test 'swap' and 'minIndex'

Sorting, as in second year university

```
class Sorter{
    void sort(List<Double> ds){
        IntStream
            .range(0,ds.size() - 1)
            .forEach(i->swap(i,minIndex(i,ds),ds));
    }
    int minIndex(int i, List<Double> ds){
        return IntStream
            .range(i + 1,ds.size())
            .reduce(i,
                (res,j)->ds.get(j) < ds.get(res) ? j : res);
    }
    void swap(int i, int j, List<Double> ds) {
        double tmp= ds.get(i);
        ds.set(i,ds.get(j));
        ds.set(j,tmp);
    }
}
```

- Statements are falling out of fashion in modern java.
- It is now much more common to use streams, as we will see later in the course.

- **Get your priorities right:**
Good division in methods is many, many times more important than having streams instead of statements.

Testing sort!

```
class TestSort {  
    //auxiliary testing helper method!!  
    void sorted(List<Double> expected, List<Double> in){  
        var res= new ArrayList<Double>(in);  
        new Sorter().sort(res);  
        assertEquals(expected,res);  
    }  
    //Many tests just using the auxiliary helper method!  
    @Test void isSorted123(){sorted(  
        List.of(1d,2d,3d),    //formatting to show clearly the expected result (first)  
        List.of(1d,2d,3d));} //and the input (second)  
  
    @Test void isInverted321(){sorted(  
        List.of(1d,2d,3d),  
        List.of(3d,2d,1d));}  
  
    @Test void isSortedEmpty(){sorted(  
        List.of(),  
        List.of());}  
  
    @Test void isSortedNegatives(){sorted(  
        List.of(-5d,-3d,-2d,-1d),    //compact formatting: important for tests,  
        List.of(-1d,-2d,-5d,-3d));} //since you should write thousands of tests  
}
```

Testing minIndex!

```
class TestSort {
    //also testing the sub methods!
    //auxiliary testing helper method!!
    void isMinIndex(int expected, List<Double> in){
        assertEquals(expected,new Sorter().minIndex(0,in));
    }
    //Many tests just using the auxiliary helper method!
    @Test void sorted123(){isMinIndex(
        0,
        List.of(1d,2d,3d));}

    @Test void inverted321(){isMinIndex(
        2,
        List.of(3d,2d,1d));}

    @Test void isEmpty(){isMinIndex(
        0, //Surprising? May be minIndex should be
        List.of());} //defined better; for example throwing an error

    @Test void negatives(){isMinIndex(
        2,
        List.of(-1d,-2d,-5d,-3d));}
}
```

Property testing

Instead of just testing that an operation on a specific input does exactly what we expect, we can generate some input and check for general properties, for example with `minIndex` we could generate random sequences of numbers and test that the number at the resulting input is indeed smaller than all the other ones.

Watch: <https://www.youtube.com/watch?v=IYzDFHx6QPY>

“The lazy programmer's guide to writing thousands of tests”
- Scott Wlaschin

This talk is also known as “The enterprise developer from hell”



**Next time:
Exceptions**

